TECHNICAL PRESENTATION

Selecting the Right ADT for E-Commerce Stock Management

How an Abstract Data Type choice drives performance efficiency for continuously updated product and stock data

Name : Bhuvanesh . S
Reg no : 192521004

The Challenge: Stock That Never Stops Changing

Massive Product Catalogs

Tens of thousands of SKUs, each with its own price, quantity, and warehouse location.

High-Frequency Updates

Every order, return, and restock changes stock counts — often hundreds of times per second.

Instant Checkout Checks

Availability must be confirmed in real time before a customer can complete a purchase.

Flash-Sale Traffic Spikes

Promotions cause sudden surges of concurrent reads and writes on the same records.

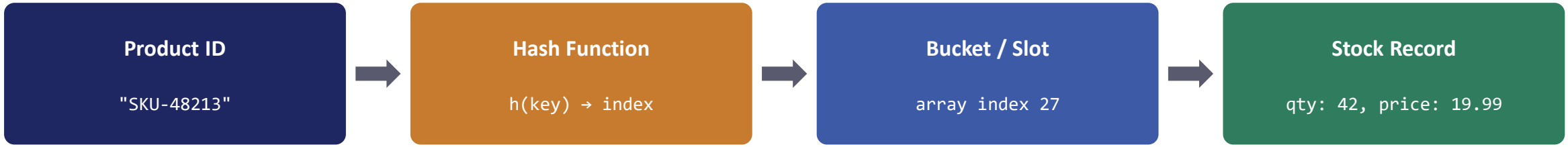
The system needs an Abstract Data Type (ADT) that supports fast lookup AND fast update of stock records at scale.

Candidate ADTs Considered

ADT	Lookup by Product ID	Update Stock	Fit for This Problem
Array / List	O(n)	O(n)	Poor — linear scan for every check
Linked List	O(n)	O(n)	Poor — no direct addressing
Binary Search Tree	O(log n)	O(log n)	Good, but slower than needed for point lookups
Hash Table (Map)	**O(1) avg**	**O(1) avg**	**Excellent — direct key access by product ID**

Hash Table stands out: constant-time average access regardless of catalog size.

How the Hash Table Resolves a Stock Lookup

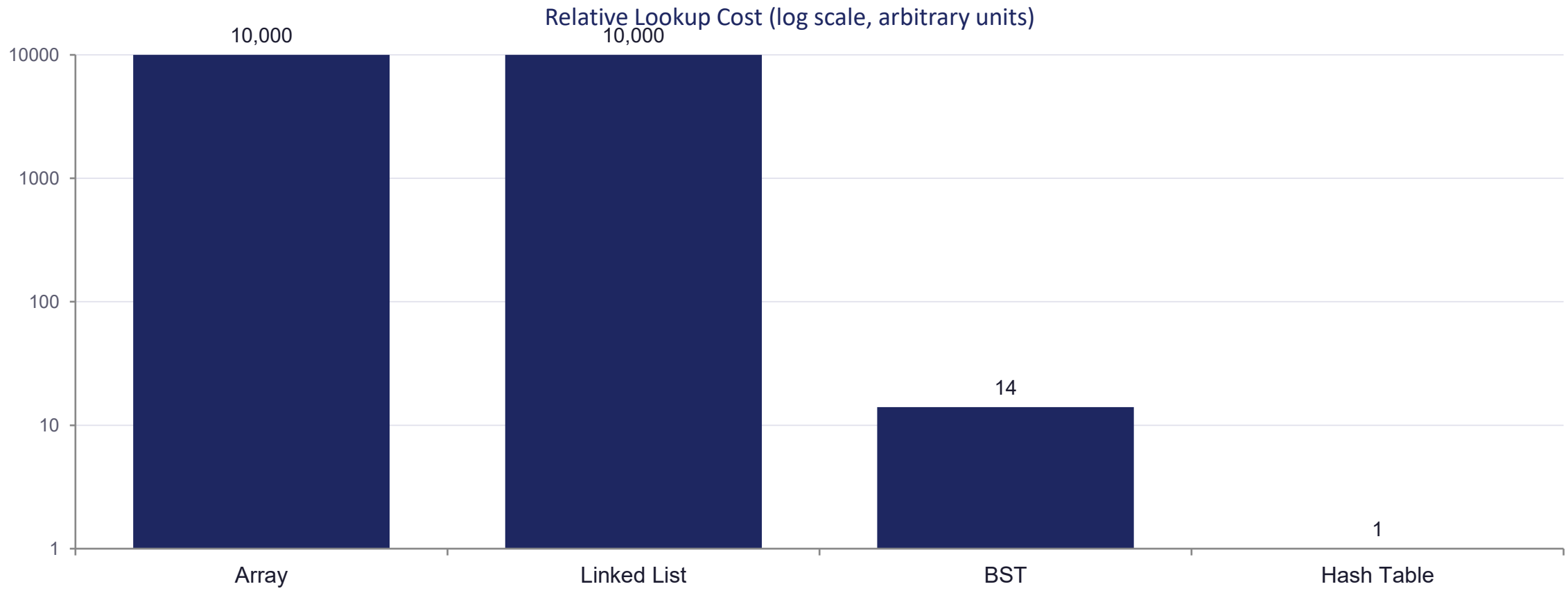


Average case: a single hash computation and one array access — independent of how many products exist.

Update in place: when an order is placed, the same lookup path finds the record and decrements quantity directly — no rewriting of the whole dataset, no shifting of other records.

Performance Efficiency: Relative Access Time

Illustrative average-case operation cost as catalog size grows (lower is faster)



Implementation Sketch

Stock is modeled as a Map ADT keyed by product ID

```
// Initialize the ADT
const stock = new Map();   // Product ID -> Stock Record

// Insert / update a product record        -> O(1) average
stock.set("SKU-48213", { qty: 50, price: 19.99, warehouse: "W3" });

// Real-time availability check at checkout -> O(1) average
function isAvailable(id, needed) {
  const record = stock.get(id);
  return record && record.qty >= needed;
}

// Apply an order, decrementing stock        -> O(1) average
function fulfillOrder(id, qty) {
  const record = stock.get(id);
  record.qty -= qty;
}
```

Handling Collisions and Scaling Up

Chaining / Open Addressing

Two product IDs that hash to the same slot are resolved with a bucket list or probing — average O(1) is preserved.

Load-Factor Monitoring

The table resizes and rehashes once it gets too full, keeping lookups fast as the catalog grows.

Sharding Across Servers

Product IDs are partitioned across multiple hash-table shards so no single server becomes a bottleneck.

Distributed Caching

Systems like Redis apply the same Map ADT concept across a cluster for high-throughput reads and writes.

Complementary Structures Alongside the Hash Table

Min-Heap / Priority Queue

Surfaces the lowest-stock items first, driving automatic restock alerts and supplier reorder priority.

Tree (B-Tree / BST)

Supports category browsing and range queries — e.g. “all products priced between $10 and $30.”

Queue

Buffers incoming order events during traffic spikes so every stock update is applied in received order.

The Hash Table remains the backbone for point lookups; these structures add the operations it does not do well.

Summary

- Recommended ADT: Hash Table (Map / Dictionary), keyed by Product ID
- O(1) average lookup and update — independent of catalog size
- Handles high-frequency stock changes without full-dataset rewrites
- Scales via chaining, resizing, sharding, and distributed caching
- Paired with heaps, trees, and queues for alerts, range queries, and ordered processing

Result: faster checkouts, accurate availability, and an inventory system that scales with the business.


Thank
you!